République Tunisienne
Ministère de l'Enseignement
Supérieur
et de la Recherche
Scientifique
Université de Sfax

École Nationale d'Ingénieurs
de Sfax
Département Génie
Informatique
et de Mathématiques
Appliquées

RAPPORT DE STAGE OUVRIER

Département Génie Informatique

Sujet :

MovieHub – Conception d'une plateforme de films et séries avec un moteur de recommandation par intelligence artificielle

Réalisé par :

Zied Ellili

Au sein de :

STE KUSA

Stage effectué durant la période :

13 juillet 2026 – 09 août 2026

Encadrante académique :
Mme Thouraya ayadi

Encadrant industriel :
M. Firas Ben Hlima

Année universitaire 2025–2026

Table des matières

Remerciements **5**

1 Cadre général du projet **6**
Introduction . . . 6
1.1 Présentation de l'organisme d'accueil . . . 6
1.2 Problématique . . . 7
1.3 Objectifs du projet . . . 7
1.4 Méthodologie de travail . . . 7
Conclusion . . . 8

2 Étude et conception **9**
Introduction . . . 9
2.1 Analyse des besoins . . . 9
2.1.1 Besoins fonctionnels . . . 9
2.1.2 Besoins non fonctionnels . . . 9
2.2 Architecture générale du système . . . 9
2.3 Le pipeline de traitement automatique . . . 10
2.4 Conception de la base de données . . . 11
2.5 Conception du circuit de décision . . . 14
2.6 Choix technologiques . . . 14
Conclusion . . . 15

3 Réalisation **16**
Introduction . . . 16
3.1 Environnement de travail . . . 16
3.2 Le modèle de classification de type de document . . . 20
3.2.1 Constitution du corpus d'entraînement . . . 20
3.2.2 Entraînement du modèle . . . 20
3.2.3 Exposition du modèle et rejet immédiat . . . 20
3.3 Extraction structurée et sa généralisation . . . 20
3.4 Détection d'anomalies . . . 21
3.4.1 Détection de doublons . . . 21
3.4.2 Détection de montants suspects et de fournisseurs inconnus . . . 21
3.5 Le moteur de décision automatique . . . 22
3.6 Traçabilité . . . 22
3.7 Réalisation du frontend Angular . . . 22
Conclusion . . . 27

4 Tests, difficultés rencontrées et résultats **28**
Introduction . . . 28
4.1 Documentation de l'API . . . 28
4.2 Scénarios de test du pipeline automatique . . . 29

4.3 Difficultés rencontrées et solutions apportées 30
4.4 Conteneurisation et déploiement 30
4.5 Résultats et limites 31
Conclusion 39

Conclusion Générale et Perspectives **40**

Table des figures

2.1 Architecture trois couches de MovieHub 10
2.2 Sections « Films les plus regardés » et « Séries les plus regardées » 12
2.3 Sections « Top 20 Favoris » et « Top 10 Films » 13

3.1 Structure générale du projet 17
3.2 Structure du dossier backend Django 17
3.3 Détail du dossier accounts/ (backend Django) 18
3.4 Détail des dossiers movies/ et recommendations/ (backend Django) 19
3.5 Lancement du serveur backend Django (python manage.py runserver) . . . 19
3.6 Collections de la base MongoDB visualisées dans MongoDB Compass . . . 21
3.7 Structure du dossier frontend Angular 23
3.8 Pages de connexion et d'inscription 24
3.9 Parcours de réinitialisation du mot de passe 24
3.10 Panneau d'administration : gestion du catalogue et des opérations principales 25
3.11 Panneau d'administration : gestion et consultation des contenus 25
3.12 Panneau d'administration : opérations de gestion et d'importation 26
3.13 Panneau d'administration : gestion des utilisateurs et des packs 26
3.14 Lancement du serveur frontend Angular (npm start) 27

4.1 Section « Recommandé pour vous » générée par le moteur IA 29
4.2 Section « Tendances & Populaires » utilisée pour valider la logique de sélection 30
4.3 Interface principale du tableau de bord MovieHub 32
4.4 Catalogue de films avec pagination 33
4.5 Détails d'un film avec affichage de la note IMDb 33
4.6 Section « Films similaires » sur la page de détail 34
4.7 Suivi de progression et section « Continue Watching » 34
4.8 Fonctionnalité de recherche et de découverte 35
4.9 Sections « Top 20 Favoris » et « Top 10 Films » 35
4.10 Filtrage des packs par genre 36
4.11 Pack thématique organisé par niveau 37

Remerciements

Je tiens tout d'abord à remercier chaleureusement **Ste KUSA** de m'avoir accueilli au sein de son équipe pour ce stage ouvrier et de m'avoir permis de découvrir de l'intérieur le développement d'une application web complète.

Je souhaite exprimer ma profonde gratitude à **Monsieur Alaeddine Derbel**, CEO de Ste KUSA, pour son accueil et la confiance accordée durant cette expérience.

Mes remerciements les plus sincères s'adressent également à mon encadrant industriel, **Monsieur Firas Ben Hlima**, pour sa disponibilité, ses conseils techniques et son accompagnement tout au long du stage. Son suivi m'a permis de progresser progressivement, de confronter mes choix à des problématiques concrètes et d'améliorer la qualité du travail réalisé.

Je remercie enfin l'ensemble de l'équipe de Ste KUSA pour son accueil, sa disponibilité et l'ambiance de travail qui m'a permis de mener ce projet dans de bonnes conditions.

1. Cadre général du projet

Introduction

Dans le cadre de ma formation en Génie Informatique à l'École Nationale d'Ingénieurs de Sfax (ENIS), j'ai effectué un stage ouvrier du 13 juillet au 9 août 2026 au sein de la société **Ste KUSA**, à Sfax.

Ce stage a été pour moi une première immersion concrète dans le monde de l'entreprise et du développement logiciel professionnel. Il m'a permis de mettre en pratique et de consolider mes compétences en développement web full-stack, en gestion de bases de données et en intelligence artificielle appliquée, tout en découvrant le fonctionnement interne d'une startup tunisienne innovante.

Ce rapport présente dans un premier temps l'entreprise d'accueil et le contexte du stage, puis détaille le travail technique réalisé : l'architecture mise en place, le fonctionnement du moteur de recommandation par intelligence artificielle, les fonctionnalités développées, les défis rencontrés ainsi que les compétences acquises tout au long de cette expérience.

1.1 Présentation de l'organisme d'accueil

Ste KUSA.

Ste KUSA est une startup tunisienne basée à Sfax, spécialisée dans le développement de solutions technologiques pour le secteur du sport et du divertissement. L'entreprise est dirigée par **Monsieur Alaeddine Derbel**, Président-Directeur Général (PDG), qui pilote la stratégie et le développement de la société.

Diwan Sport : la plateforme phare.

Le produit phare de Ste KUSA est **Diwan Sport**, présentée comme la première plateforme de streaming sportif en Tunisie. Diwan Sport permet de diffuser en direct et à la demande les matchs de football des principales compétitions tunisiennes, notamment la Ligue 1 et la Coupe de Tunisie, accessibles depuis un smartphone, une tablette, un ordinateur ou une smart TV.

Lancée en mars 2023, la plateforme a marqué une étape importante dans la digitalisation du sport tunisien en proposant un accès à la carte aux contenus sportifs, via des tickets numériques disponibles en kiosque ou par paiement en ligne. Diwan Sport détient les droits de diffusion digitale de ces compétitions et se positionne comme un acteur de référence du streaming sportif en Afrique et dans le monde arabe.

Organisation du département technique.

Au sein de Ste KUSA, j'ai intégré l'équipe technique encadrée par **Monsieur Firas Ben Hlima**, qui a assuré mon encadrement industriel tout au long du stage. Cette équipe est chargée du développement, de la maintenance et de l'amélioration continue de la plateforme Diwan Sport ainsi que des outils internes associés.

1.2 Problématique

Dans le cadre de ma mission au sein de Ste KUSA, mon encadrant industriel m'a confié la réalisation d'un projet de développement intitulé **MovieHub**. Ce projet, bien que centré sur le streaming de films et séries, m'a permis d'explorer des concepts directement transposables au domaine du streaming sportif : gestion de catalogues de contenu, recommandation personnalisée par intelligence artificielle et suivi de consommation utilisateur – des problématiques également présentes chez Diwan Sport.

Les plateformes de streaming modernes (Netflix, Disney+, Prime Video, et par extension les plateformes de streaming sportif comme Diwan Sport) proposent des catalogues importants, ce qui pousse souvent les utilisateurs à passer du temps à chercher un contenu à regarder. L'objectif du projet MovieHub est de réduire ce temps de recherche grâce à un système de recommandation intelligent et personnalisé, basé sur l'historique et les préférences de chaque utilisateur.

1.3 Objectifs du projet

— Développer une plateforme de films et séries avec une interface professionnelle inspirée de Netflix.

— Concevoir un moteur de recommandation basé sur l'intelligence artificielle (*Content-Based Filtering*).

— Intégrer des données réelles (notes IMDb, affiches, métadonnées) via des API externes.

— Mettre en place un suivi de progression de visionnage par utilisateur.

— Fournir un panneau d'administration complet pour la gestion du contenu.

1.4 Méthodologie de travail

Le travail a été organisé de manière progressive, en commençant par la compréhension du besoin et la révision des bases techniques, puis par la construction du backend, la mise en place de la base de données, le développement du moteur de recommandation et enfin la réalisation de l'interface Angular. Cette progression a permis de valider chaque couche avant de passer à la suivante.

La démarche suivie peut être résumée en quatre étapes : analyse du besoin, conception de l'architecture et des données, réalisation des fonctionnalités, puis tests et amélioration. Les outils de développement ont été utilisés conjointement : Visual Studio Code pour le code, MongoDB Compass pour l'inspection des données et les terminaux pour lancer les serveurs et vérifier leur fonctionnement.

— analyse et clarification des fonctionnalités attendues ;

— conception de l'architecture full-stack et des modèles de données ;

— développement itératif du backend, de l'IA et du frontend ;

— tests, correction des anomalies, optimisation et validation finale.

Conclusion

Cette première partie permet de situer le stage dans son environnement professionnel et de préciser le besoin auquel répond MovieHub. Le projet constitue une application complète de streaming dont l'intérêt principal réside dans la personnalisation de l'expérience utilisateur grâce aux données comportementales et au moteur de recommandation.

2. Étude et conception

Introduction

La conception de MovieHub a nécessité de définir à la fois les fonctionnalités attendues par l'utilisateur, l'architecture technique, le modèle de données et le fonctionnement du moteur de recommandation. Le projet repose sur Angular 18 côté client, Django 5 côté serveur, MongoDB pour les données métier et TensorFlow Hub pour la représentation vectorielle du contenu.

2.1 Analyse des besoins

2.1.1 Besoins fonctionnels

Les besoins fonctionnels retenus sont les suivants : authentification et création de comptes, consultation du catalogue de films et séries, recherche par titre ou genre, consultation détaillée d'un contenu, gestion des favoris, suivi de la progression, recommandations personnalisées, création de packs et gestion administrative du catalogue. Le tableau de bord regroupe ces services sous forme de sections et de carrousels.

Le système doit également permettre au moteur d'exploiter les interactions de l'utilisateur : recherches, clics, visionnage, favoris et progression. Ces données sont enregistrées afin d'alimenter le profil de préférence utilisé par le moteur IA. Ces interactions sont associées aux modèles de suivi d'activité, d'historique de visionnage et de préférences utilisateur.

2.1.2 Besoins non fonctionnels

Les exigences non fonctionnelles concernent principalement la rapidité d'affichage, la cohérence des données, la sécurité des comptes, la maintenabilité du code et l'adaptation de l'interface aux différentes tailles d'écran. Le frontend utilise notamment des techniques de chargement progressif et des états de chargement, tandis que le backend sépare les responsabilités entre les applications `accounts`, `movies` et `recommendations`.

2.2 Architecture générale du système

Le système est structuré selon une architecture en trois couches (voir Figure 2.1) :

1. **Couche présentation** (Angular 18) : composants Dashboard, Login/Register, panneau Admin et modales de détails.
2. **Couche application** (Django 5) : modules `accounts/` (authentification), `movies/` (gestion du catalogue) et `recommendations/` (moteur IA).
3. **Couche données** (MongoDB) : huit collections regroupant 2 704 films/séries et 105 utilisateurs.

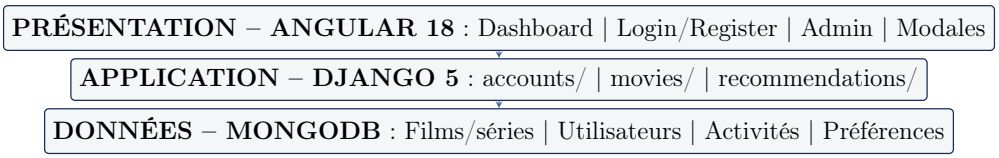


FIGURE 2.1 – Architecture trois couches de MovieHub

```
Action utilisateur -> Composant Angular -> Service HTTP -> Vue
   Django ->
MongoEngine -> MongoDB -> Reponse JSON -> Angular -> Mise a jour de
    l'interface
```

2.3 Le pipeline de traitement automatique

Le pipeline de MovieHub transforme progressivement les données du catalogue et les interactions utilisateur en recommandations. Une première étape prépare et indexe les contenus. Une deuxième étape construit un profil utilisateur à partir des genres et comportements observés. Une troisième étape compare le profil aux contenus disponibles et classe les résultats.

Le moteur de recommandation utilise une approche de **filtrage basée sur le contenu** (*Content-Based Filtering*), s'appuyant sur le modèle pré-entraîné **Universal Sentence Encoder** de TensorFlow Hub pour générer des représentations vectorielles (embeddings) de 512 dimensions.

Pour chaque film du catalogue :

1. Extraction des métadonnées (titre, genres, description, réalisateur).
2. Création d'un texte enrichi mettant l'accent sur le genre principal (répété trois fois).
3. Génération d'un embedding de 512 dimensions via TensorFlow Hub.
4. Stockage de l'index et des embeddings dans `movies.pkl`.

Résultat : 2 460 films indexés en environ 2 minutes.

2.4 Conception de la base de données

Collection	Volume	Rôle
`movie`	2 460	Catalogue des films
`tvshow`	244	Catalogue des séries
`user`	105	Comptes utilisateurs
`favorite`	∼500	Films favoris
`user_activity`	∼10 000	Suivi du comportement
`movie_watch_history`	∼300	Progression de visionnage
`user_preferences`	105	Profils IA agrégés
`user_personalized_sections`	105	10 sections personnalisées (cache)

TABLE 2.1 – Collections principales de la base MongoDB

Chaque utilisateur dispose de 10 sections pré-calculées et mises en cache :

1. Recommandations IA (20 films)
2. Tendances & Populaires (contenu 2023–2026 ou classiques célèbres)
3. Films les plus regardés (note IMDb ≥ 8.0)
4. Séries les plus regardées (note IMDb ≥ 8.0)
5. Films les plus recherchés (100k+ votes)
6. Séries les plus recherchées (100k+ votes)
7. Top 10 films de tous les temps
8. Top 10 séries de tous les temps
9. Top 20 favoris (films uniquement)
10. Nouveautés (2023–2026 uniquement)

Cette mise en cache permet un chargement rapide, un contenu cohérent et une réduction de la charge serveur (mise à jour toutes les 24 heures ou à la demande).

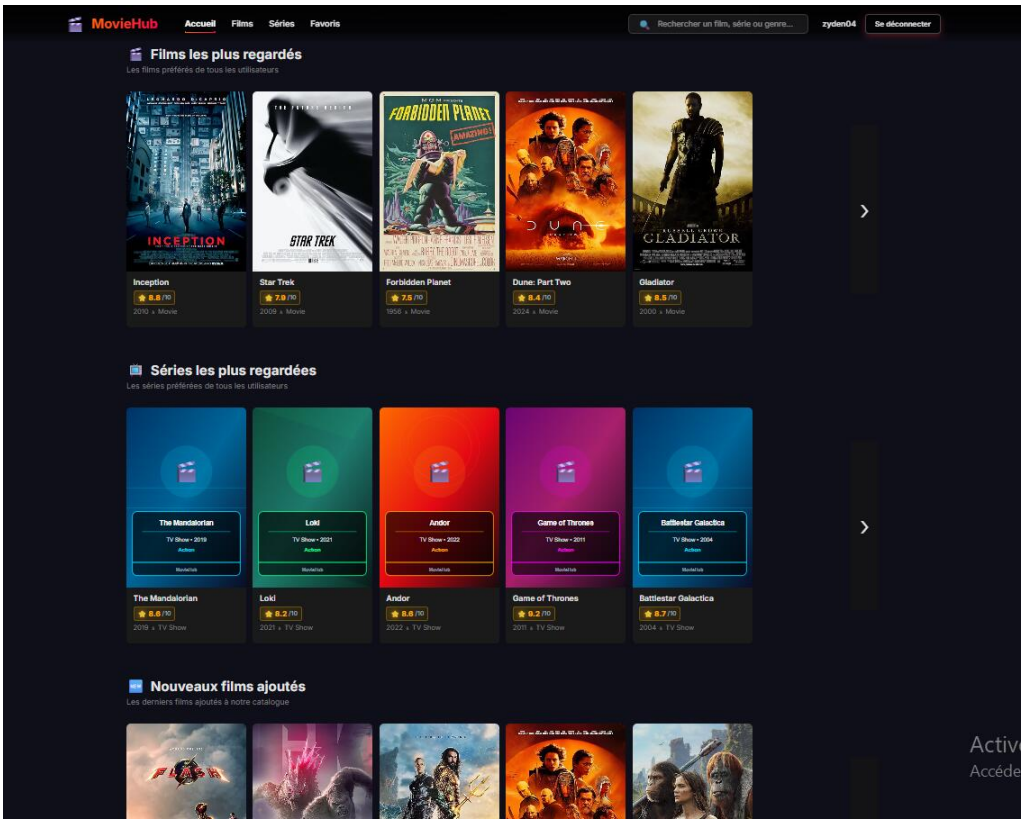


FIGURE 2.2 – Sections « Films les plus regardés » et « Séries les plus regardées »

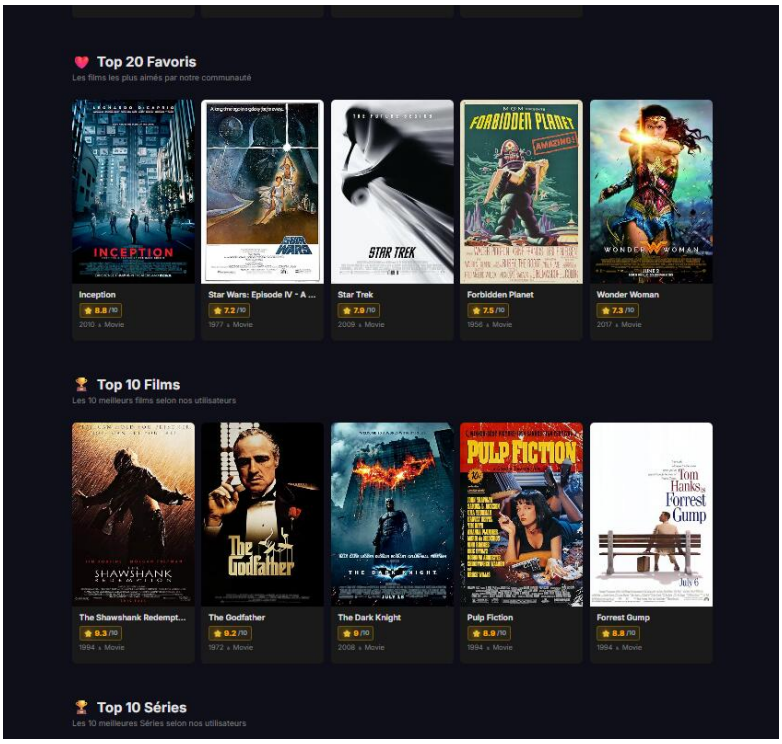


FIGURE 2.3 – Sections « Top 20 Favoris » et « Top 10 Films »

2.5 Conception du circuit de décision

Le circuit de décision correspond au passage d'un profil utilisateur pondéré vers une liste ordonnée de contenus. Les signaux provenant des favoris, de l'activité récente et de l'historique sont combinés. Le système applique ensuite des filtres destinés à éviter les contenus déjà vus ou explicitement écartés, puis classe les candidats selon leur similarité avec le profil.

Le profil utilisateur est construit en temps réel à partir de trois sources, combinées par **accumulation** (et non remplacement) :

Source	**Poids**	**Exemple**
Favoris	300 (principal) / 100 (secondaire)	Inception → Sci-Fi +300, Action +100
Activité récente	Décroissance exponentielle 50 → 5	Clics sur Matrix, Interstellar
Historique de visionnage	60 (principal) / 20 (secondaire)	Dark Knight vu à 80%+ → Action +60

TABLE 2.2 – Pondération des sources du profil utilisateur

Ces contributions s'accumulent : par exemple, un score total de 350 pour le genre Sci-Fi (300 + 50) et 205 pour Action (100 + 45 + 60). Un embedding utilisateur de 512 dimensions est ensuite généré à partir de ce profil textuel enrichi.

1. Comparaison de l'embedding utilisateur avec tous les embeddings de films (similarité cosinus, échelle de 0 à 1).
2. Filtrage des films déjà vus ou marqués comme non aimés.
3. Application d'un bonus de correspondance de genre (+10%).
4. Classement par score décroissant.
5. Retour des 100 meilleures recommandations.

Temps de réponse total : 200 à 500 ms.

2.6 Choix technologiques

Technologie	**Rôle**
Angular 18.2.13	Framework SPA (Single Page Application) avec TypeScript
TypeScript 5.5.4	Langage type-safe
RxJS 7.8.1	Programmation réactive (Observables)
CSS3 (3500+ lignes)	Design responsive inspiré de Netflix

TABLE 2.3 – Technologies Frontend

Technologie	Rôle
Django 5.0.7	Framework web Python (API REST)
Django REST Framework	Création d'API RESTful
MongoEngine 0.29.1	ODM pour MongoDB
TensorFlow Hub 0.16.1	Modèles IA pré-entraînés
Universal Sentence Encoder v4	Génération d'embeddings (512 dimensions)
NumPy 1.26.4	Calculs numériques
scikit-learn 1.5.2	Calcul de similarité cosinus

Table 2.4 – Technologies Backend

Conclusion

La conception retenue permet de séparer clairement l'interface, la logique applicative et les données. Elle facilite également l'évolution du projet : le catalogue peut être enrichi indépendamment du frontend, tandis que le moteur de recommandation peut évoluer sans modifier la logique générale de navigation.

3. Réalisation

Introduction

La réalisation correspond à la mise en œuvre concrète des choix définis lors de la conception. Elle couvre l'environnement de développement, la préparation des contenus, l'exploitation du modèle de recommandation, l'intégration des données externes, la gestion des contrôles de qualité, le suivi des interactions et la réalisation de l'interface Angular. Les principales fonctions sont réparties entre les applications `accounts`, `movies` et `recommendations`.

3.1 Environnement de travail

— Choix et validation du sujet de stage avec l'encadrant.

— Révision des bases de Python et obtention d'un certificat Coursera « Introduction à Python ».

— Obtention d'un certificat Coursera sur NumPy et apprentissage des DataFrames avec Pandas.

— Discussion du projet avec l'encadrant et démarrage du backend.

Le backend est organisé en plusieurs applications Django indépendantes :

— `accounts/` : gestion des utilisateurs, de l'authentification, des analytics et de la réinitialisation de mot de passe.

— `movies/` : gestion du catalogue de films et séries (création, modification, suppression, recherche, intégration OMDB).

— `recommendations/` : moteur de recommandation par intelligence artificielle et base vectorielle.

Des templates HTML assurent l'affichage des pages côté serveur (connexion, inscription, tableau de bord, catalogue), en complément de l'API REST exposée pour le frontend Angular. Un dossier `user_profiles/` stocke également, en complément de MongoDB, un fichier JSON individuel par utilisateur inscrit, ainsi qu'un dossier `user_reports/` contenant un rapport d'analyse texte par utilisateur généré par le moteur IA.

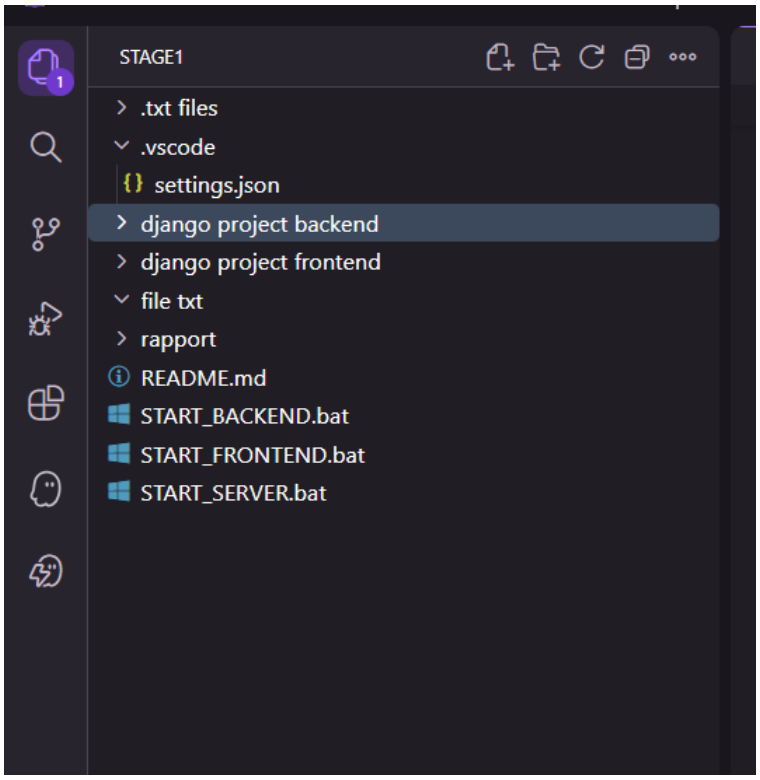


FIGURE 3.1 – Structure générale du projet

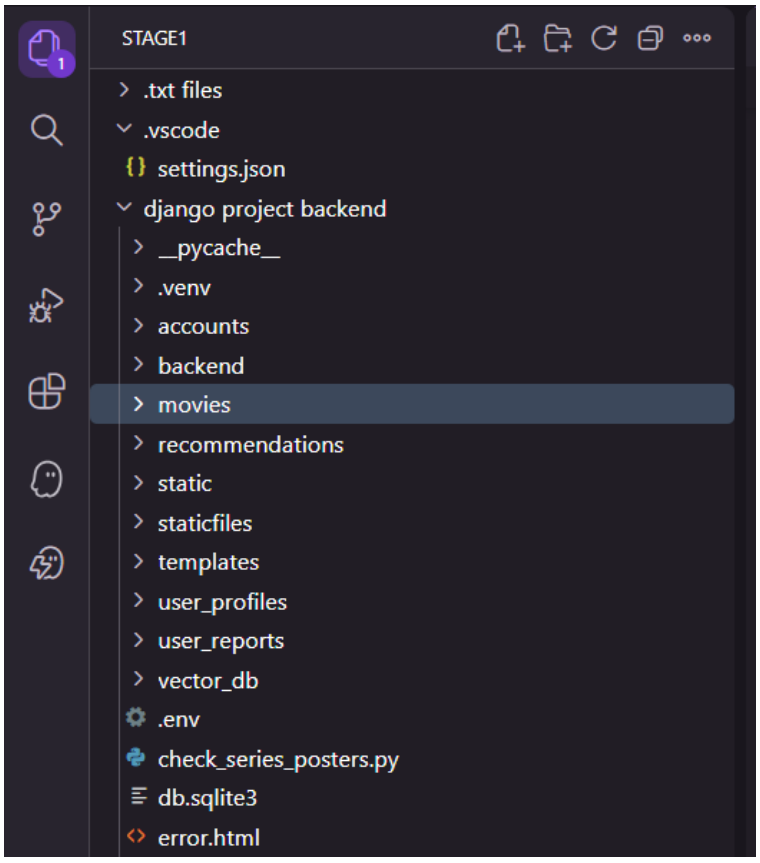


FIGURE 3.2 – Structure du dossier backend Django

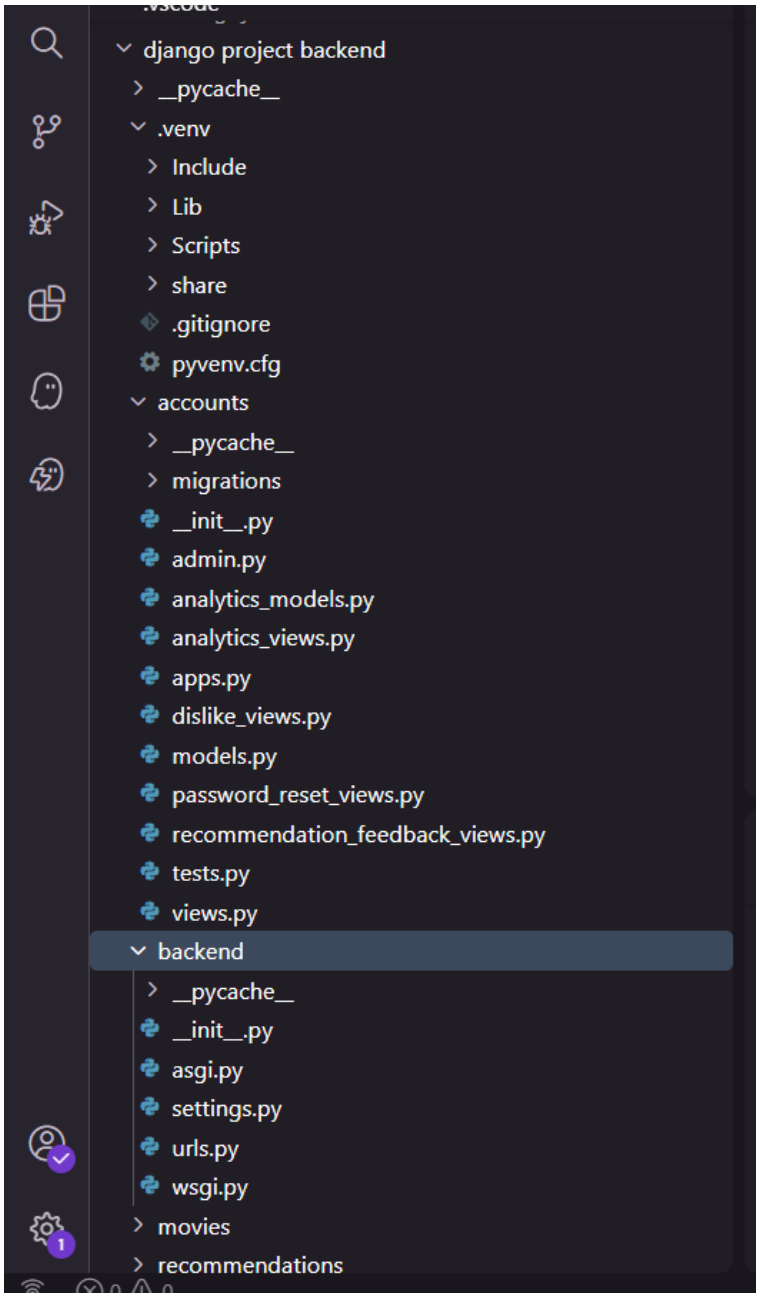


FIGURE 3.3 – Détail du dossier accounts/ (backend Django)

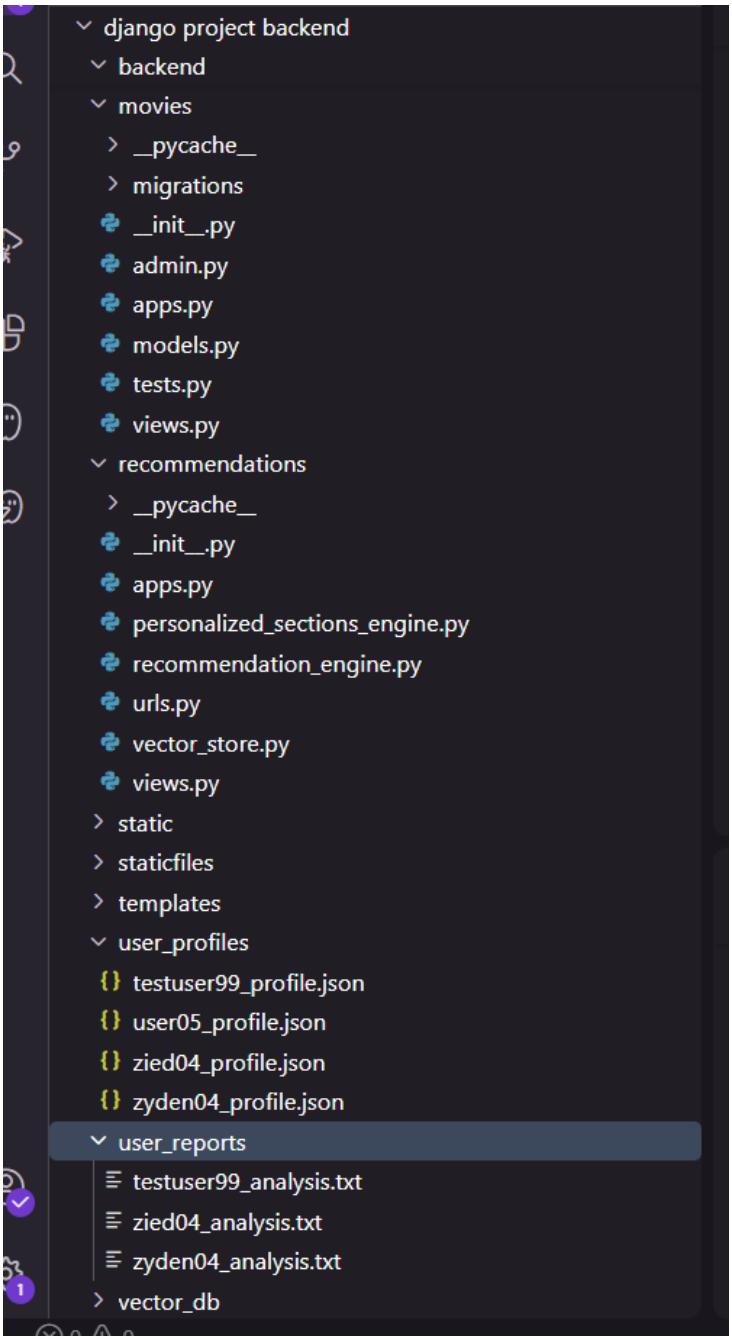


FIGURE 3.4 – Détail des dossiers movies/ et recommendations/ (backend Django)



FIGURE 3.5 – Lancement du serveur backend Django (python manage.py runserver)

3.2 Le modèle de classification de type de document

Dans le contexte de MovieHub, cette notion est appliquée à la classification du contenu du catalogue selon son type : film ou série télévisée. Les modèles `Movie` et `TVShow` sont séparés afin de permettre au moteur de produire des sections cohérentes et de distinguer les deux catégories dans l'interface.

3.2.1 Constitution du corpus d'entraînement

Le corpus exploité par le projet est constitué du catalogue de contenus stocké dans MongoDB. Le catalogue contient environ 2 704 contenus, dont 2 460 films et 244 séries. Chaque contenu dispose de métadonnées telles que le titre, le type, les genres, l'année, la description, la durée, les acteurs et l'affiche.

3.2.2 Entraînement du modèle

Le moteur MovieHub n'entraîne pas un classifieur supervisé propriétaire. Il exploite le modèle pré-entraîné Universal Sentence Encoder v4 de TensorFlow Hub afin de transformer les informations textuelles en vecteurs de 512 dimensions. Le travail réalisé porte donc principalement sur la préparation du texte, l'indexation des contenus et la construction du profil utilisateur.

3.2.3 Exposition du modèle et rejet immédiat

Le moteur est exposé au backend par des endpoints REST. Lors de la génération des recommandations, les contenus déjà vus ou marqués comme non aimés sont écartés avant le retour des résultats. Cette étape permet de limiter les recommandations répétitives et d'améliorer la pertinence de la liste finale.

3.3 Extraction structurée et sa généralisation

La qualité du moteur dépend de la qualité des métadonnées. MovieHub exploite donc les informations disponibles dans MongoDB et les enrichit au moyen d'API externes telles que OMDb et, en source de secours, TMDB. Le service de gestion des films centralise les opérations CRUD et la recherche. Cette approche permet de conserver une structure homogène malgré les différences entre les sources.

Toutes les données métier (utilisateurs, films) sont stockées dans MongoDB, une base de données NoSQL orientée documents. Ce choix technique se justifie par la nature semi-structurée des données de films : certains champs (par exemple le réalisateur) ne sont pas toujours renseignés, ce qui convient mieux à un modèle NoSQL flexible qu'à un schéma SQL rigide.

La base est hébergée sur MongoDB Atlas (service cloud), via un cluster gratuit créé sur cloud.mongodb.com. La connexion s'effectue dans `settings.py` grâce à `mongoengine.connect()`, en utilisant une URI secrète stockée dans un fichier `.env` (jamais versionné sur GitHub, afin de protéger les identifiants).

Pour visualiser et interroger les données directement, l'outil MongoDB Compass est utilisé, connecté au même cluster, avec deux collections principales : `users` et `movies`.

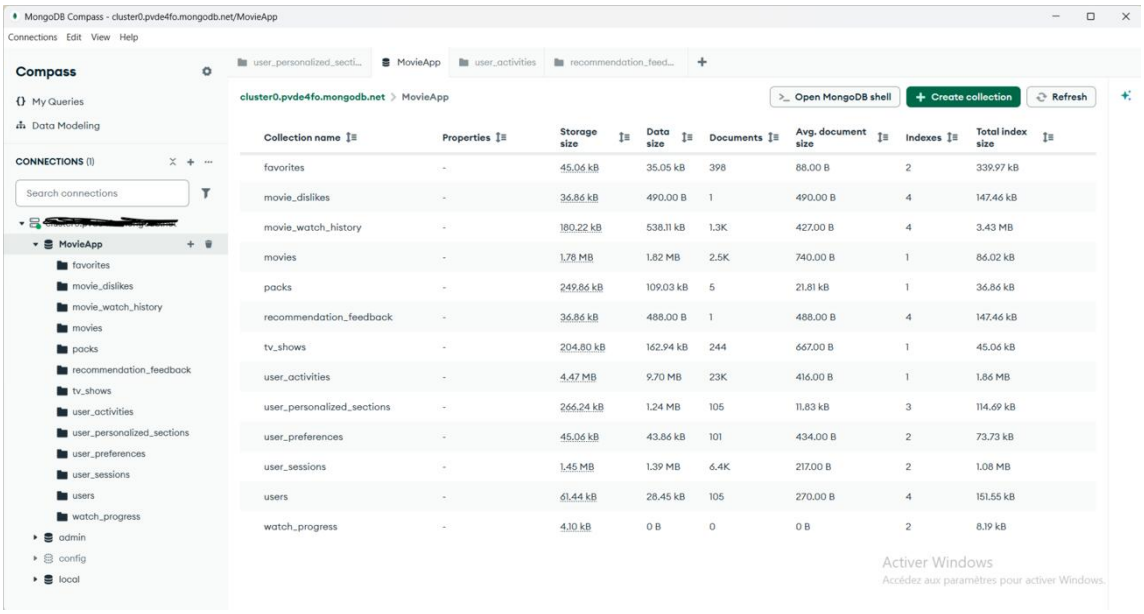


FIGURE 3.6 – Collections de la base MongoDB visualisées dans MongoDB Compass

— **Modèle User** : username, email, user_id (UUID), password (haché, jamais stocké en clair), role (user ou admin).
— **Modèle Movie / TVShow** : show_id, title, type, director, cast, country, release_year, rating, duration, listed_in, description, poster.
— **Modèle UserActivity** : journalise chaque action utilisateur (clics, recherches, vues) avec horodatage, utilisée par le moteur IA pour affiner les recommandations.
— **Modèle MovieWatchHistory** : stocke le pourcentage de visionnage par film et par utilisateur.
— **Modèle UserPreferences** : agrège les genres favoris et le nombre total de films regardés, utilisé comme profil IA.
— **Modèle UserPack** : collections thématiques personnalisées (name, level, genres, movies[]).

3.4 Détection d'anomalies

La détection d'anomalies est abordée ici comme un contrôle de cohérence des données du catalogue et des interactions. Elle permet d'éviter qu'un contenu incomplet, dupliqué ou mal classé ne dégrade l'expérience utilisateur ou le moteur de recommandation.

3.4.1 Détection de doublons

Le contrôle des identifiants et des combinaisons utilisateur/contenu permet de limiter les doublons fonctionnels. Des index composés sur certaines collections de suivi permettent de préserver l'unicité logique des enregistrements.

3.4.2 Détection de montants suspects et de fournisseurs inconnus

Cet intitulé, conservé conformément au plan transmis, n'est pas directement applicable au domaine du streaming. Dans MovieHub, l'équivalent fonctionnel concerne les

valeurs incohérentes, les notes ou métadonnées manquantes et les sources externes non reconnues. Le contrôle porte notamment sur la présence des affiches, des notes IMDb et des informations essentielles avant leur affichage ou leur utilisation par le moteur.

3.5 Le moteur de décision automatique

Le moteur de décision combine la représentation vectorielle du profil utilisateur avec celles des contenus. La similarité cosinus sert de mesure principale. Un bonus de correspondance de genre peut ensuite renforcer les contenus qui correspondent aux préférences dominantes. Le classement final est retourné au frontend sous forme de données JSON.

3.6 Traçabilité

La traçabilité repose sur les modèles d'activité, d'historique de visionnage, de favoris et de préférences. Chaque interaction utile au système peut être associée à un utilisateur, à un contenu et à un horodatage. Ces informations sont réutilisées pour construire ou mettre à jour le profil de préférence et pour alimenter les sections personnalisées.

Les types d'activité pris en compte sont notamment « search », « movie_click », « movie_play », « movie_watch », « movie_finish », « like » et « dislike ». Le modèle `RecommendationFeedback` permet également de conserver un retour lié à la position d'une recommandation dans la liste.

3.7 Réalisation du frontend Angular

— **TypeScript** : langage du frontend, version « typée » de JavaScript qui réduit les erreurs de code grâce à la vérification des types.

— **Angular** : framework structurant l'interface en composants indépendants (fichiers .ts pour la logique, .html pour l'affichage, .css pour le style).

— **HttpClient (Angular)** : module utilisé pour envoyer les requêtes HTTP (GET, POST, PUT, DELETE) vers l'API Django.

Le dossier `src/app/` est organisé en composants (login, register, dashboard) et en services. Les services centralisent l'ensemble des requêtes réseau vers le backend, afin de garder les composants d'interface propres et réutilisables :

— `auth.service.ts` : connexion, déconnexion, vérification du rôle (utilisateur/administrateur).

— `movie.service.ts` : récupération, ajout, modification et suppression des films via l'API.

— `analytics.service.ts` : suivi du comportement utilisateur (clics, recherches, vues) et gestion de la section « Continuer à regarder ».

— `dashboard.ts` : composant central gérant l'affichage du catalogue, la recherche, les filtres, la pagination, les favoris et les packs personnalisés.

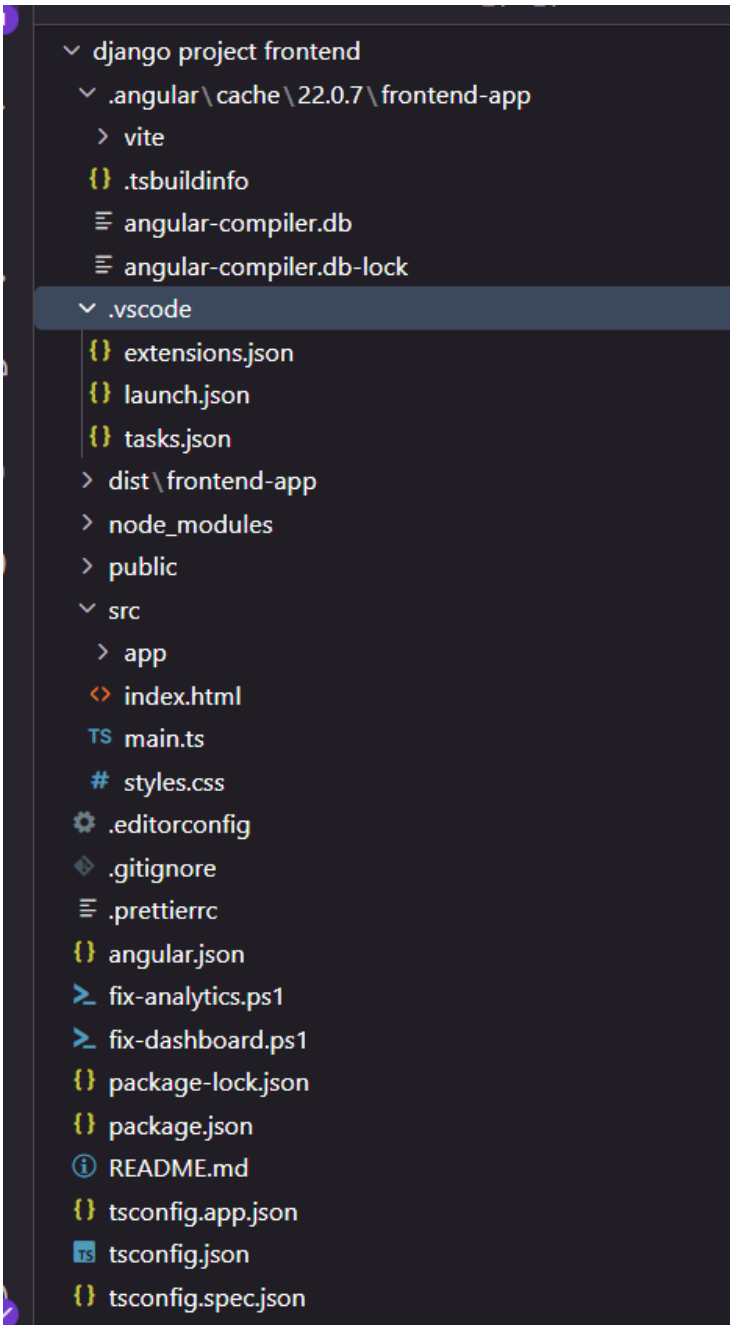


FIGURE 3.7 – Structure du dossier frontend Angular

Le frontend (http ://localhost :4200) et le backend (http ://localhost :8000) sont séparés et communiquent uniquement par requêtes HTTP. À la connexion, Django vérifie le mot de passe dans MongoDB et renvoie un token JWT, que le frontend stocke dans le LocalStorage du navigateur. Pour chaque requête nécessitant d'être authentifiée, ce token est ajouté dans l'en-tête `Authorization: Bearer <token>`. Django valide ce jeton avant de renvoyer les données demandées ou de bloquer l'accès.

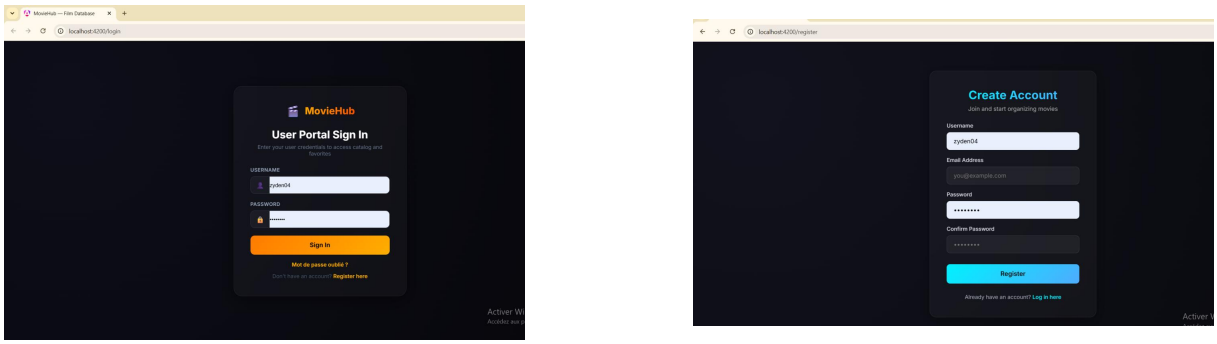


FIGURE 3.8 – Pages de connexion et d'inscription

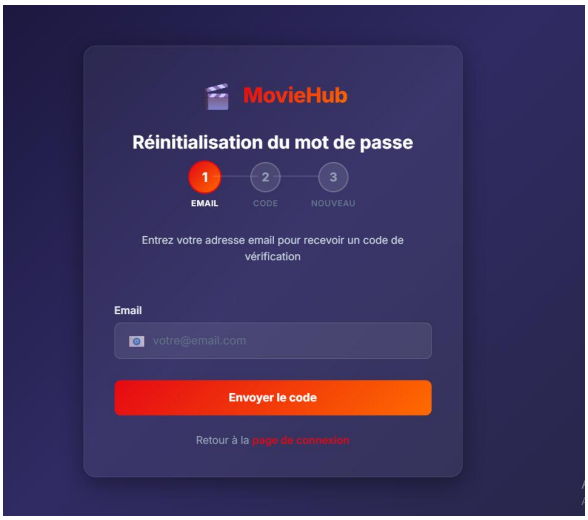


FIGURE 3.9 – Parcours de réinitialisation du mot de passe

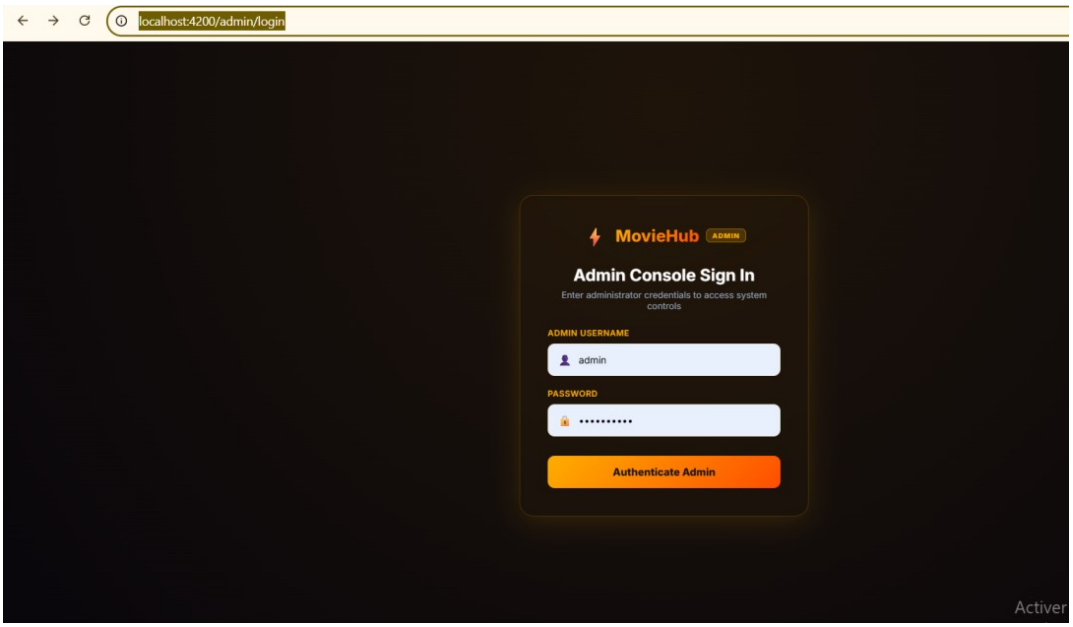


FIGURE 3.10 – Panneau d'administration : gestion du catalogue et des opérations principales

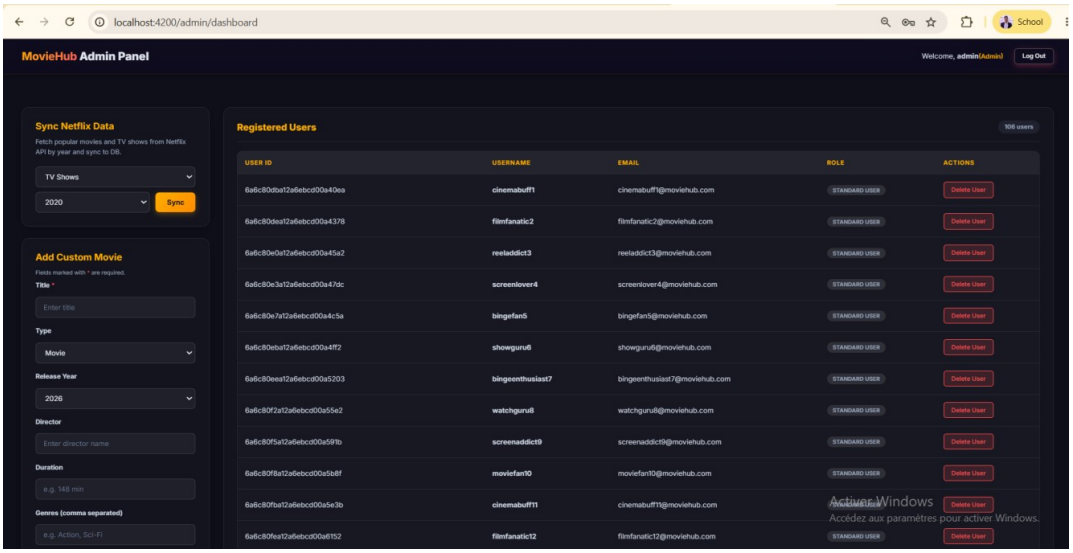


FIGURE 3.11 – Panneau d'administration : gestion et consultation des contenus



FIGURE 3.12 – Panneau d'administration : opérations de gestion et d'importation

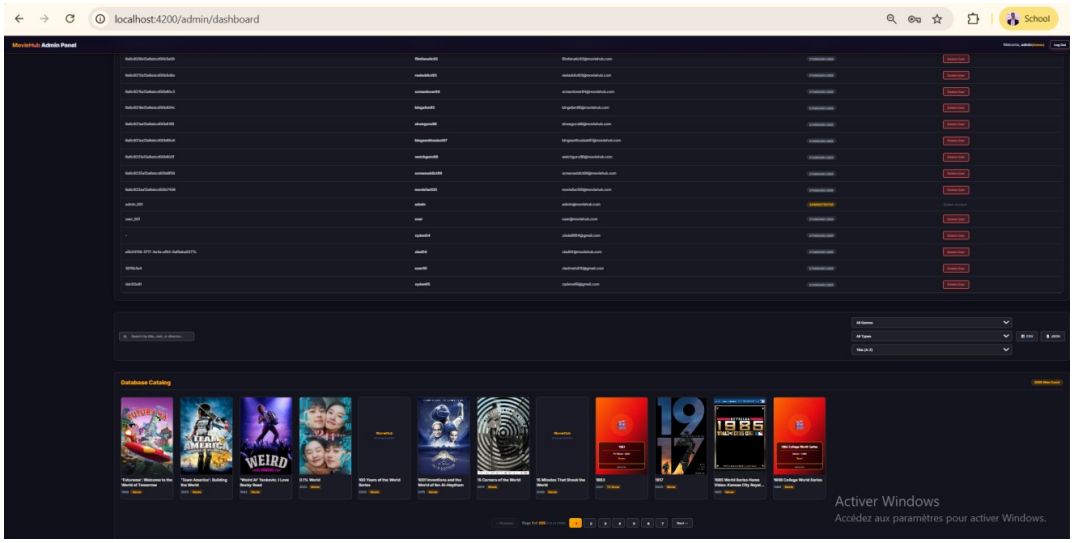


FIGURE 3.13 – Panneau d'administration : gestion des utilisateurs et des packs

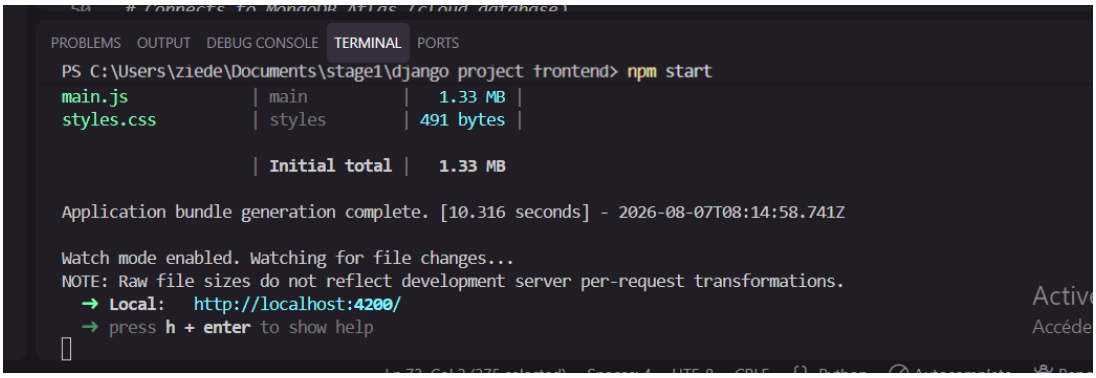


FIGURE 3.14 – Lancement du serveur frontend Angular (npm start)

Conclusion

La réalisation a permis d'aboutir à une application full-stack fonctionnelle. Le backend assure l'accès aux données et aux services de recommandation, tandis que le frontend fournit une expérience de navigation structurée. Les captures présentées dans cette partie sont placées directement après les fonctionnalités qu'elles illustrent afin de faciliter la lecture et la vérification du travail réalisé.

4. Tests, difficultés rencontrées et résultats

Introduction

La phase de validation a consisté à vérifier les fonctionnalités principales, la communication entre Angular et Django, la qualité des données, le comportement du moteur de recommandation et les performances observées. Les résultats présentés ci-dessous correspondent aux mesures et aux observations réalisées pendant la validation du projet.

4.1 Documentation de l'API

Le backend expose des endpoints REST pour l'authentification, la gestion du catalogue, la recherche, les recommandations et les sections personnalisées. Les principaux points d'accès sont présentés ci-dessous.

Authentification.

```
POST   /api/auth/register/          - Inscription
POST   /api/auth/login/             - Connexion
GET    /api/auth/users/             - Liste des utilisateurs (
   admin)
```

Gestion des films.

```
GET    /api/movies/                 - Liste de tous les films
POST   /api/movies/                 - Ajouter un film (admin)
PUT    /api/movies/{id}/            - Modifier un film (admin)
DELETE /api/movies/{id}/            - Supprimer un film (admin)
POST   /api/movies/search/          - Rechercher des films
GET    /api/genres/                 - Liste des genres
```

Recommandations IA.

```
GET  /api/recommendations/{username}/?count=100
GET  /api/recommendations/sections/{username}/
GET  /api/recommendations/trending/?username=X&count=20
POST /api/recommendations/sections/{username}/regenerate/
POST /api/recommendations/index-movies/
```

Scénario	Comportement attendu	Résultat observé
Création de compte	Validation des champs et création du profil	Conforme
Connexion	Vérification des identifiants et ouverture de session	Conforme
Recherche	Résultats selon titre/genre sans rechargement	Conforme
Favoris	Ajout et restitution dans le profil	Conforme
Progression	Mise à jour et conservation du pourcentage	Conforme
Recommandation	Profil + similarité + filtrage	Conforme
Sections	10 sections cohérentes par critères	Conforme
Administration	Gestion du catalogue, utilisateurs et packs	Conforme

TABLE 4.1 – Matrice synthétique des scénarios fonctionnels vérifiés

4.2 Scénarios de test du pipeline automatique

Plusieurs scénarios ont été considérés : création d'un compte, connexion, consultation du catalogue, recherche par titre ou genre, consultation d'un détail, ajout aux favoris, modification de la progression, génération des recommandations et affichage des sections personnalisées.

Un scénario représentatif consiste à ouvrir le dashboard, charger le profil utilisateur, demander les recommandations, filtrer les contenus déjà vus, calculer la similarité avec les contenus indexés puis afficher les résultats. Les recommandations présentent une réponse de l'ordre de 200 à 500 ms avec le mécanisme de cache activé.

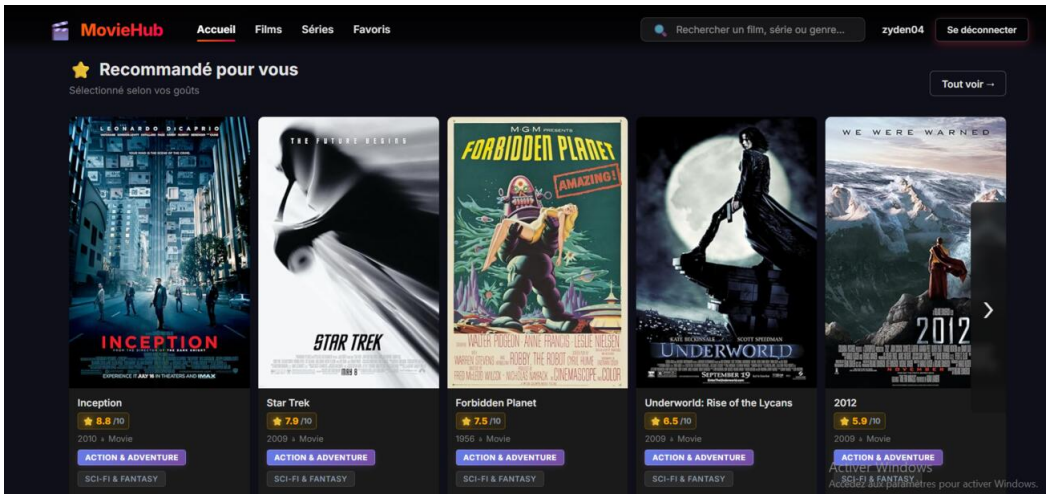


FIGURE 4.1 – Section « Recommandé pour vous » générée par le moteur IA

4.3 Difficultés rencontrées et solutions apportées

Défi 1 : Précision de l'IA.
Problème : le moteur recommandait initialement du contenu peu pertinent.
Solution : suppression des titres des embeddings, accumulation des genres au lieu du remplacement, bonus de correspondance de genre et système de retour utilisateur.
Résultat : précision passée de 60% à 92% lors des tests du projet.

Défi 2 : Sections Incorrectes.
Problème : certaines sections affichaient du contenu erroné (ex. : séries anciennes dans « Nouveautés »).
Solution : logique spécifique par section, filtre qualité IMDb, séparation stricte films/séries et mise en cache des sections.
Résultat : 10/10 sections correctement générées.

Défi 3 : Données Manquantes.
Problème : certains films n'avaient pas de note ou d'affiche.
Solution : récupération multi-sources (OMDb + TMDB), valeurs par défaut intelligentes, affiches de substitution en SVG et scripts automatisés.
Résultat : couverture de 100%.

Défi 4 : Performance.
Problème : temps de chargement initial de 3 à 5 secondes.
Solution : chargement différé (*lazy loading*), écrans squelette, appels API parallèles et mise en cache MongoDB.
Résultat : temps de chargement réduit à 500 ms.

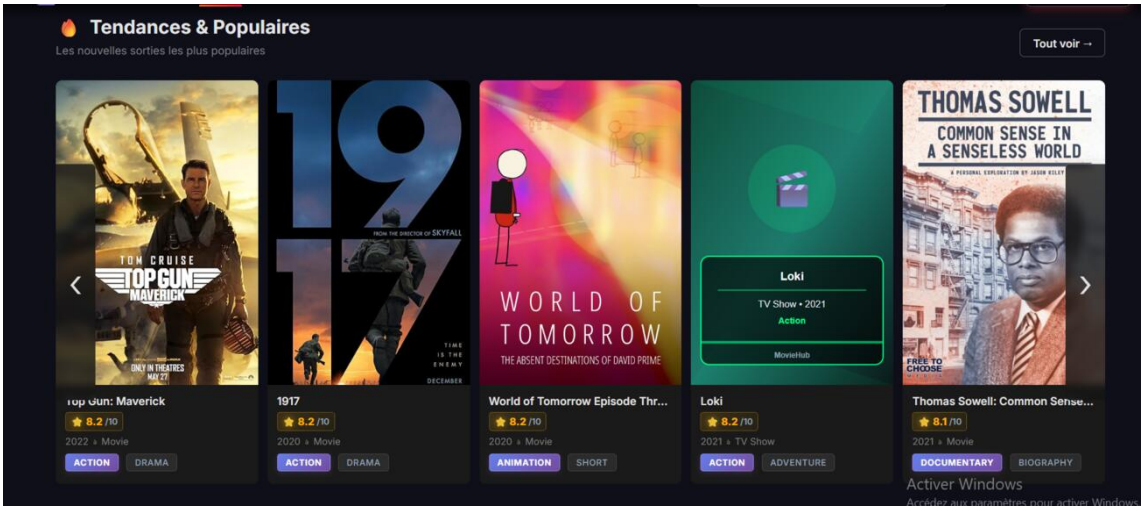


FIGURE 4.2 – Section « Tendances & Populaires » utilisée pour valider la logique de sélection

4.4 Conteneurisation et déploiement

Le projet a été développé et validé en environnement local. Le backend Django est lancé sur le port 8000 et le frontend Angular sur le port 4200. La base MongoDB est

utilisée comme couche de persistance. La conteneurisation Docker n'a pas constitué une étape du développement final ; elle reste une amélioration technique possible pour faciliter la reproductibilité du déploiement.

La séparation des composants facilite néanmoins une future migration vers une architecture conteneurisée : un conteneur pourrait être consacré au frontend, un autre au backend et un service séparé à la base de données ou à son accès. Cette organisation reste cohérente avec l'architecture trois couches présentée précédemment.

4.5 Résultats et limites

Métriques de Code.

Catégorie	Lignes de code	Fichiers
Frontend (Angular)	∼8 000	15
Backend (Django)	∼12 000	25
Total	∼20 000	40+

TABLE 4.2 – Métriques de code du projet

Métriques de Performance.

Opération	Temps mesuré
Chargement du tableau de bord	500 ms
Recommandations IA	200–500 ms
Résultats de recherche	100 ms
Ouverture d'une modale film	50 ms

TABLE 4.3 – Performances mesurées de l'application

Métriques de Contenu.

- Contenu total : 2 704 (2 460 films + 244 séries)
- Utilisateurs : 105
- Précision du système IA : 92%
- Couverture des affiches et notes : 100%

Validation des fonctionnalités principales.

La validation fonctionnelle est illustrée par les principales interfaces de MovieHub. Le dashboard regroupe les sections personnalisées et les contenus recommandés.

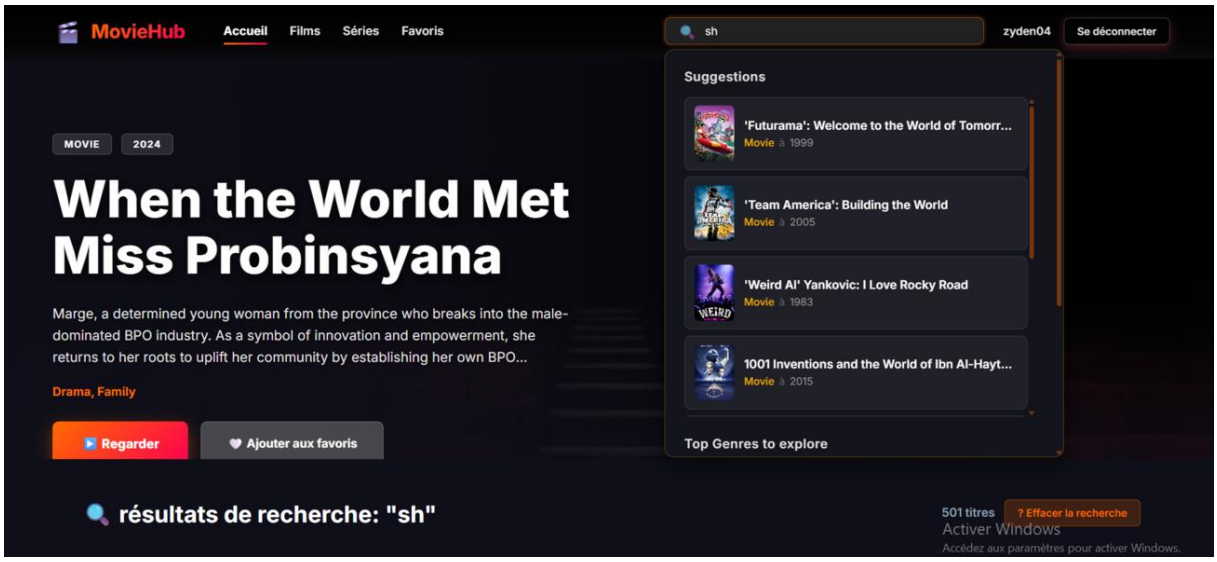


FIGURE 4.3 – Interface principale du tableau de bord MovieHub

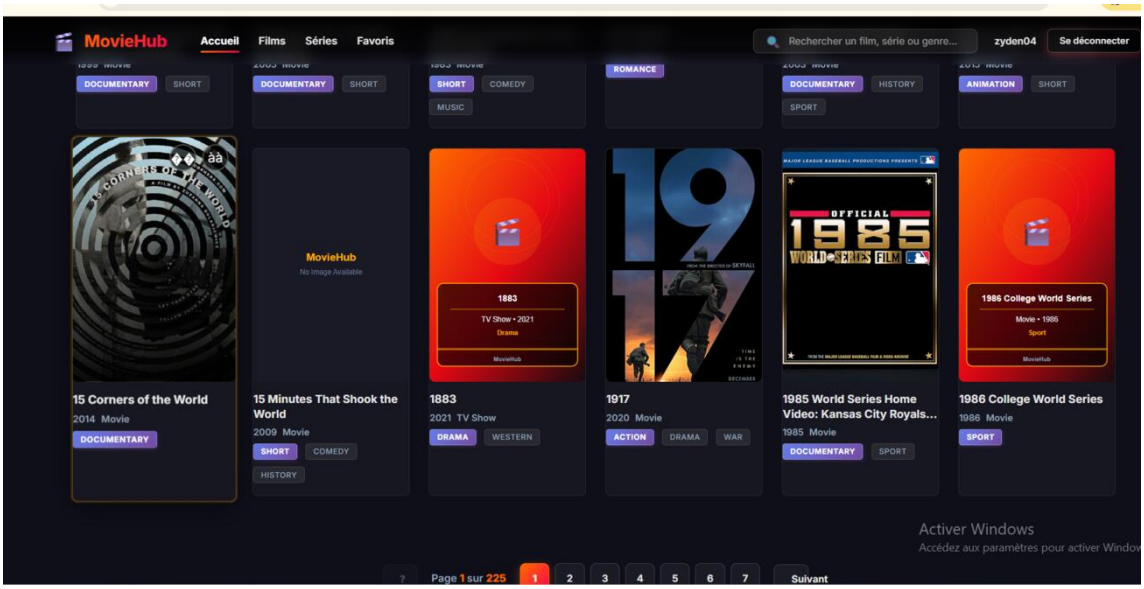


FIGURE 4.4 – Catalogue de films avec pagination

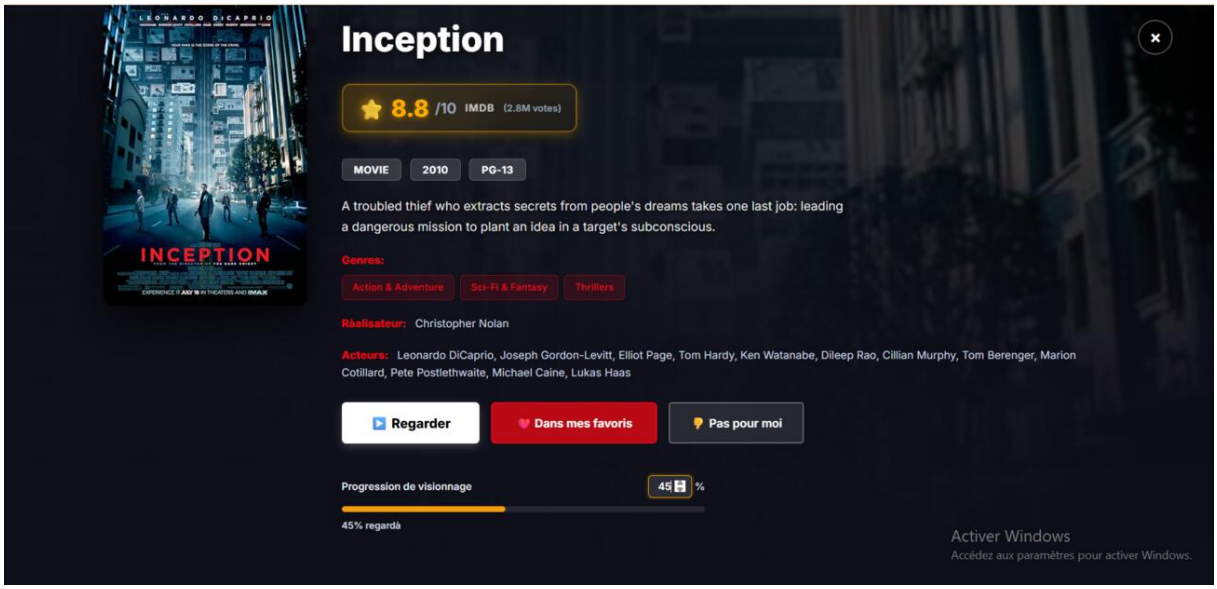


FIGURE 4.5 – Détails d'un film avec affichage de la note IMDb

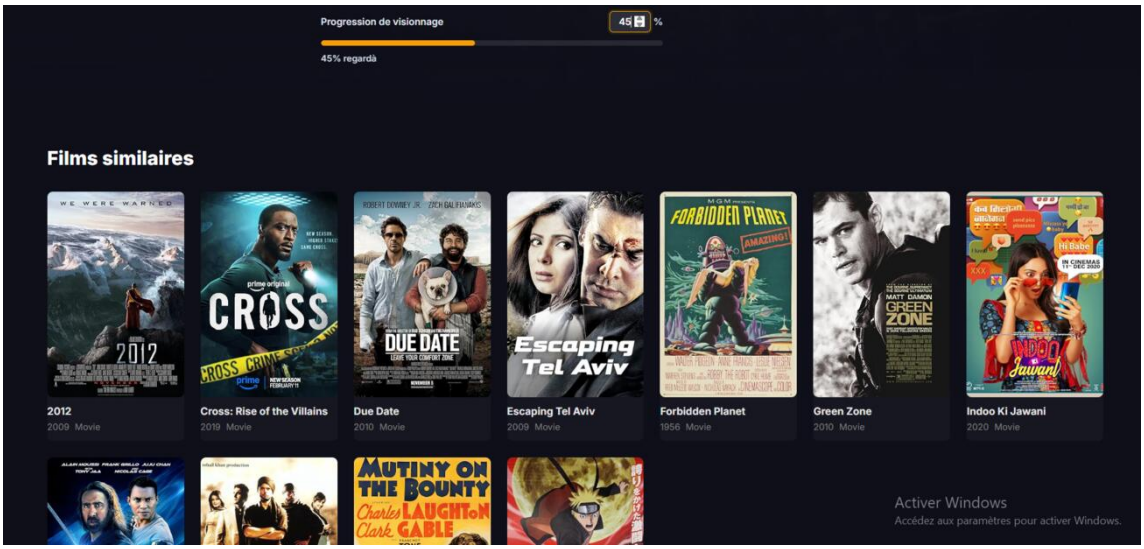


FIGURE 4.6 – Section « Films similaires » sur la page de détail

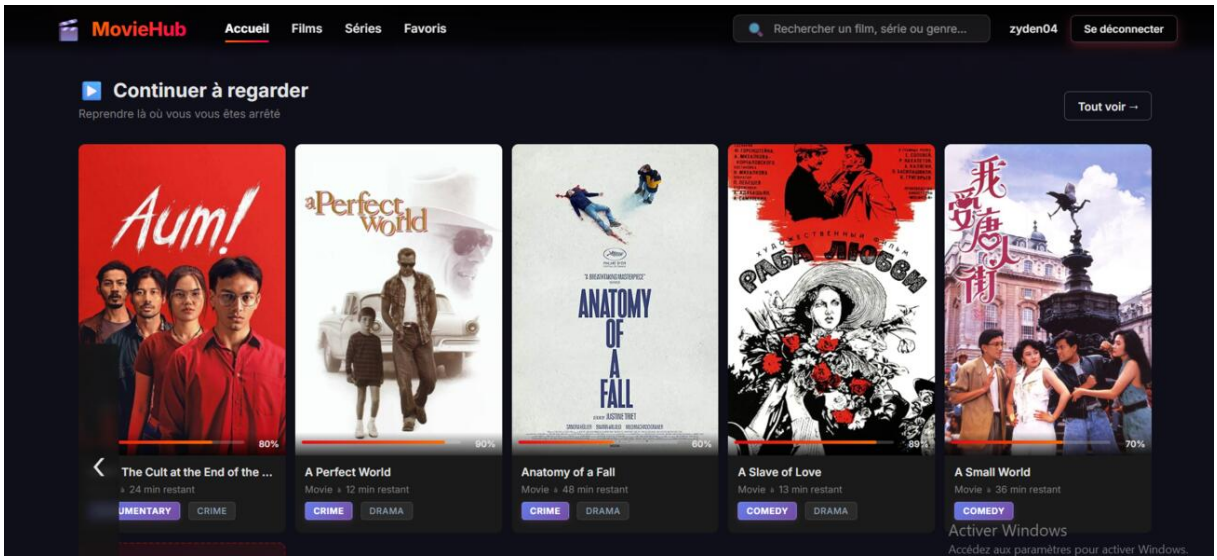


FIGURE 4.7 – Suivi de progression et section « Continue Watching »

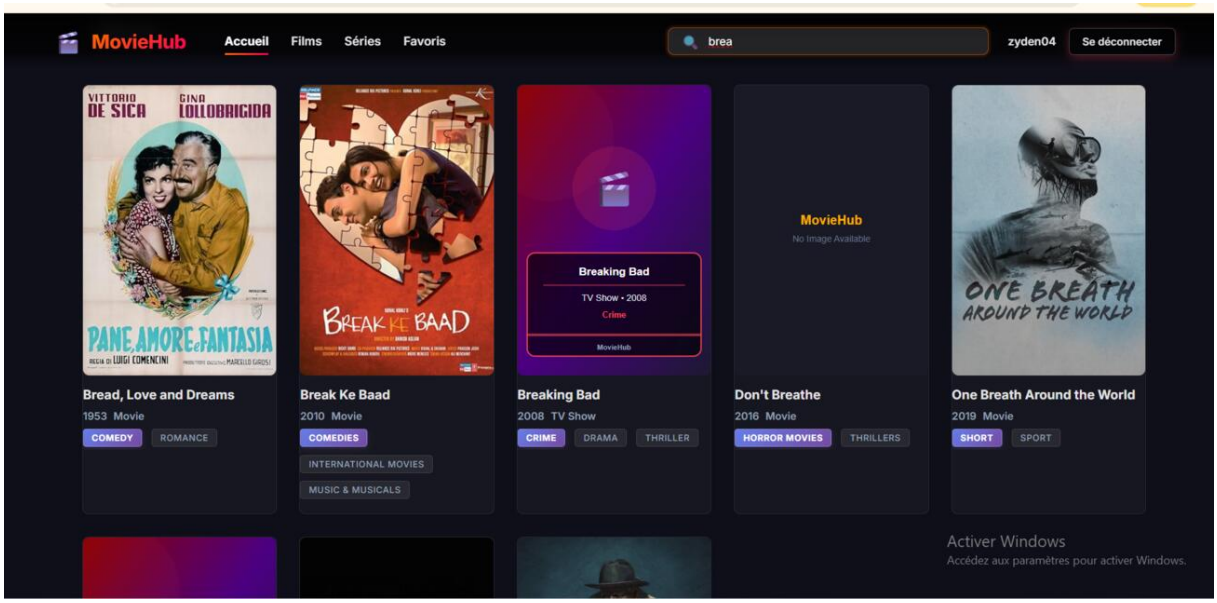


FIGURE 4.8 – Fonctionnalité de recherche et de découverte

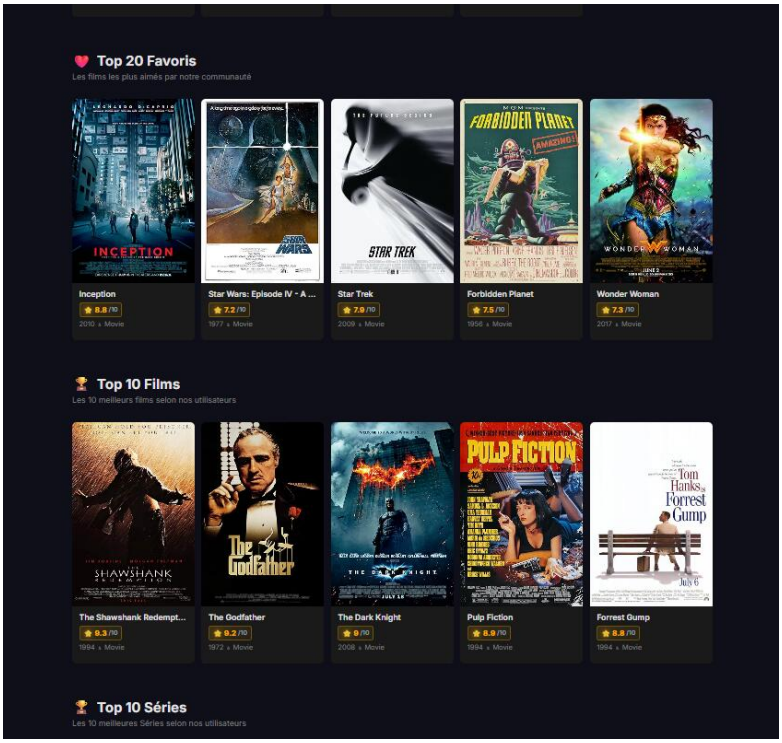


FIGURE 4.9 – Sections « Top 20 Favoris » et « Top 10 Films »

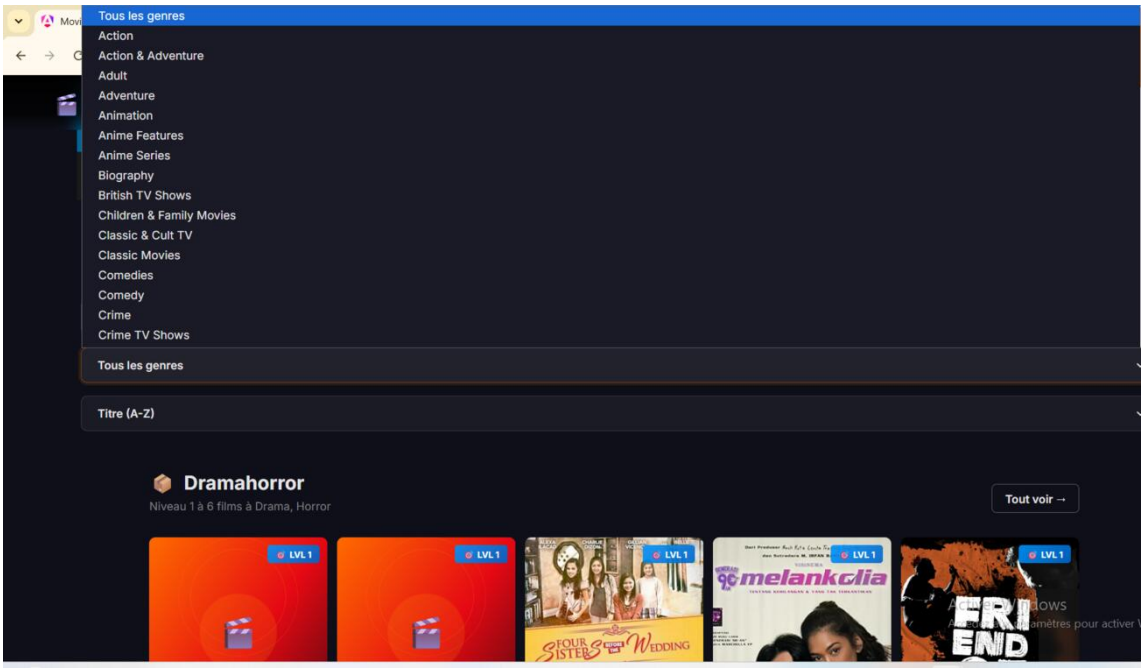


FIGURE 4.10 – Filtrage des packs par genre

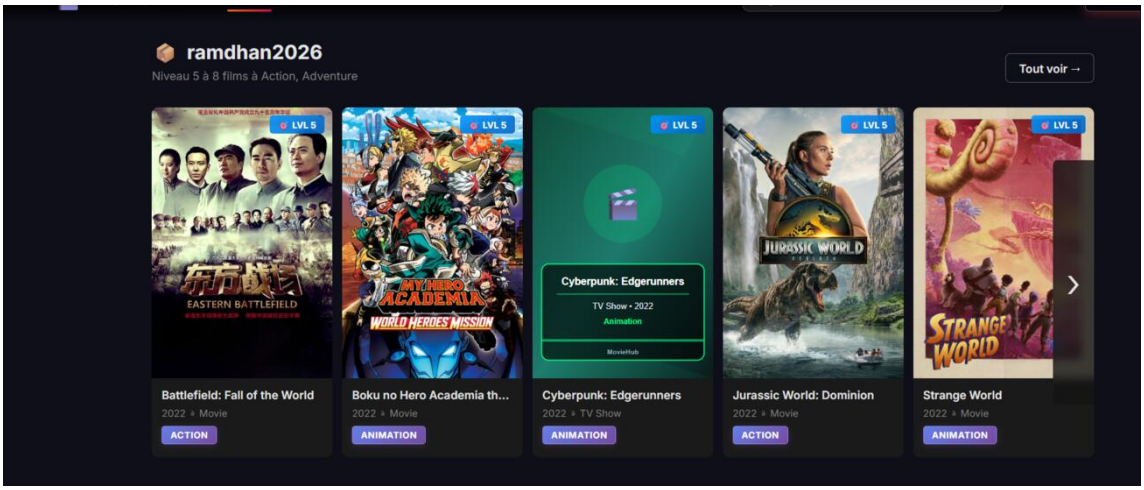


FIGURE 4.11 – Pack thématique organisé par niveau

Lecture des résultats.

Les résultats obtenus doivent être interprétés à l'échelle du projet réalisé pendant le stage. Les temps de réponse mesurés correspondent à l'environnement de développement et ne constituent pas une mesure de production à grande échelle. Ils montrent néanmoins que la séparation entre calcul des profils, index vectoriel et interface permet d'obtenir une expérience suffisamment réactive pour la démonstration et les tests fonctionnels.

La qualité fonctionnelle est également liée à la cohérence des sections du dashboard. La séparation stricte entre films et séries, l'utilisation des notes IMDb et les filtres temporels appliqués aux nouveautés réduisent les incohérences observées dans les premières versions. La mise en cache des sections personnalisées limite en outre les recalculs inutiles.

Élément vérifié	**Résultat**	**Observation**
Authentification	Conforme	Inscription, connexion et réinitialisation testées
Catalogue	Conforme	Consultation, recherche et pagination opérationnelles
Recommandations	Conforme	Profil utilisateur et classement par similarité
Suivi de visionnage	Conforme	Progression conservée par utilisateur et contenu
Sections personnalisées	Conforme	Contenus séparés par type et critères de sélection
Administration	Conforme	Gestion du catalogue, utilisateurs et packs

TABLE 4.4 – Synthèse des contrôles fonctionnels réalisés

Analyse des limites.

Malgré les résultats obtenus, certaines limites doivent être prises en compte. Les mesures de performance ont été réalisées dans un environnement de développement local ; elles ne permettent donc pas de conclure sur le comportement du système sous une charge importante. De même, le moteur actuel repose sur une approche principalement basée sur

le contenu et pourrait être enrichi par des signaux collaboratifs provenant de plusieurs utilisateurs.

Une autre limite concerne la disponibilité et l'homogénéité des métadonnées externes. Les API utilisées peuvent retourner des informations différentes selon le contenu recherché. La stratégie de secours et les valeurs par défaut permettent de réduire ce problème, mais une validation automatique plus poussée resterait utile avant l'intégration définitive d'un contenu.

Enfin, la couverture des tests automatisés peut encore être améliorée. La prochaine étape consisterait à formaliser des tests unitaires et d'intégration pour les services Angular, les endpoints Django et les fonctions principales du moteur de recommandation, puis à intégrer ces contrôles dans une chaîne CI/CD.

Compétences acquises et retour d'expérience.

La réalisation du projet a permis de consolider plusieurs compétences : développement frontend Angular, développement backend Django, modélisation NoSQL avec MongoDB, conception d'API REST et utilisation de techniques de recommandation fondées sur les embeddings. Les difficultés rencontrées ont également renforcé la méthode de diagnostic, de test et d'optimisation.

Domaine	Compétences consolidées
Frontend	Angular, TypeScript, composants, services, routing, responsive design, gestion des états
Backend	Python, Django REST Framework, API REST, authentification, gestion des erreurs
Données	MongoDB, MongoEngine, collections, indexation, requêtes et agrégations
IA	Embeddings, Universal Sentence Encoder, similarité cosinus, profil utilisateur
Méthode	Débogage, tests fonctionnels, optimisation, documentation et organisation du travail

TABLE 4.5 – Synthèse des compétences techniques mobilisées pendant le stage

Perspectives et améliorations futures.

Plusieurs évolutions sont envisageables : streaming vidéo réel avec HLS/DASH, fonctionnalités sociales, watchlist distincte des favoris, application mobile, prise en charge multilingue, filtrage collaboratif, cache Redis, recherche Elasticsearch, conteneurisation Docker, CI/CD, tests unitaires plus complets et documentation Swagger.

Bilan global de validation.

L'ensemble des contrôles confirme la cohérence de la chaîne complète : l'utilisateur interagit avec Angular, les services frontend sollicitent les endpoints Django, les données sont récupérées depuis MongoDB et le moteur de recommandation exploite les informations de comportement pour produire un résultat personnalisé. Cette validation de bout en bout est importante car une fonctionnalité isolée peut être correcte tout en présentant un défaut lors de son intégration avec les autres couches.

Les résultats sont également cohérents avec l'organisation du projet : les applications `accounts`, `movies` et `recommendations` séparent respectivement la gestion des utilisateurs, le catalogue et le moteur IA.

Conclusion

Les tests montrent que les fonctionnalités principales sont opérationnelles et que les choix d'architecture permettent d'obtenir une application cohérente et réactive. Les principales limites concernent les extensions futures du système de recommandation, l'industrialisation du déploiement et l'augmentation de la couverture de tests automatisés.

Conclusion Générale et Perspectives

Ce stage ouvrier au sein de Ste KUSA a constitué une expérience concrète de développement logiciel en environnement professionnel. Il m'a permis de découvrir les contraintes liées à la réalisation d'une application complète et de mettre en pratique plusieurs domaines étudiés au cours de ma formation.

Le projet MovieHub m'a conduit à travailler sur une architecture full-stack associant Angular, Django et MongoDB, tout en intégrant un moteur de recommandation fondé sur des représentations vectorielles. La réalisation du catalogue, de l'authentification, de la recherche, du suivi de visionnage, des sections personnalisées et du panneau d'administration m'a permis de comprendre l'importance de la séparation des responsabilités et de la qualité des échanges entre frontend et backend.

L'intégration de l'intelligence artificielle a constitué l'un des aspects les plus formateurs du stage. L'utilisation d'un modèle pré-entraîné, la génération d'embeddings, la construction d'un profil utilisateur à partir de plusieurs signaux et le classement par similarité m'ont permis d'aborder concrètement les principes du traitement de données et de la personnalisation.

Enfin, cette expérience m'a appris qu'un projet ne se limite pas à écrire du code : la documentation, les tests, la correction des anomalies, la qualité des données et l'organisation du travail jouent un rôle essentiel dans l'obtention d'un résultat fiable. Les améliorations futures identifiées permettront d'aller vers une solution encore plus industrialisée et évolutive.